

AY 2025-26

# Data Analytics with R

## Quality Laboratory Manual

### Experiment No. 06

To implement layered plots using ggplot2 to create aesthetics and themes



**Course Instructor –**  
**Dr. Tahseen A. Mulla**

## **Experiment No. 06**

**Title of Experiment:** To implement layered data visualization using ggplot with aesthetics and themes

**Aim of Experiment:** To implement layered plots in R using the ggplot2 package and customize visualizations using aesthetics mappings and themes

### **System Requirements –**

- Operating System: Windows 10 or above
- RAM: Minimum 4 GB
- Hard disk: 10 GB free space
- Processor: 2.33 GHz or higher

### **Software/s Needed for Experiment**

- R programming language (latest stable version)
- RStudio IDE
- Spreadsheet software (Microsoft Excel/ Libre Office)
- Built-in R graphics package
- ggplot2 package
- tidyverse package
- Internet Connection (for dataset access)/ CSV dataset file

### **Experiment Objectives:**

- To understand layered grammar of graphics
- To use ggplot() with aesthetic mappings
- To add multiple layers using geom\_\*() functions
- To customize plots using colors, shapes and sizes
- To apply built-in theme
- To modify titles, labels and legends

### **Experiment Outcomes:**

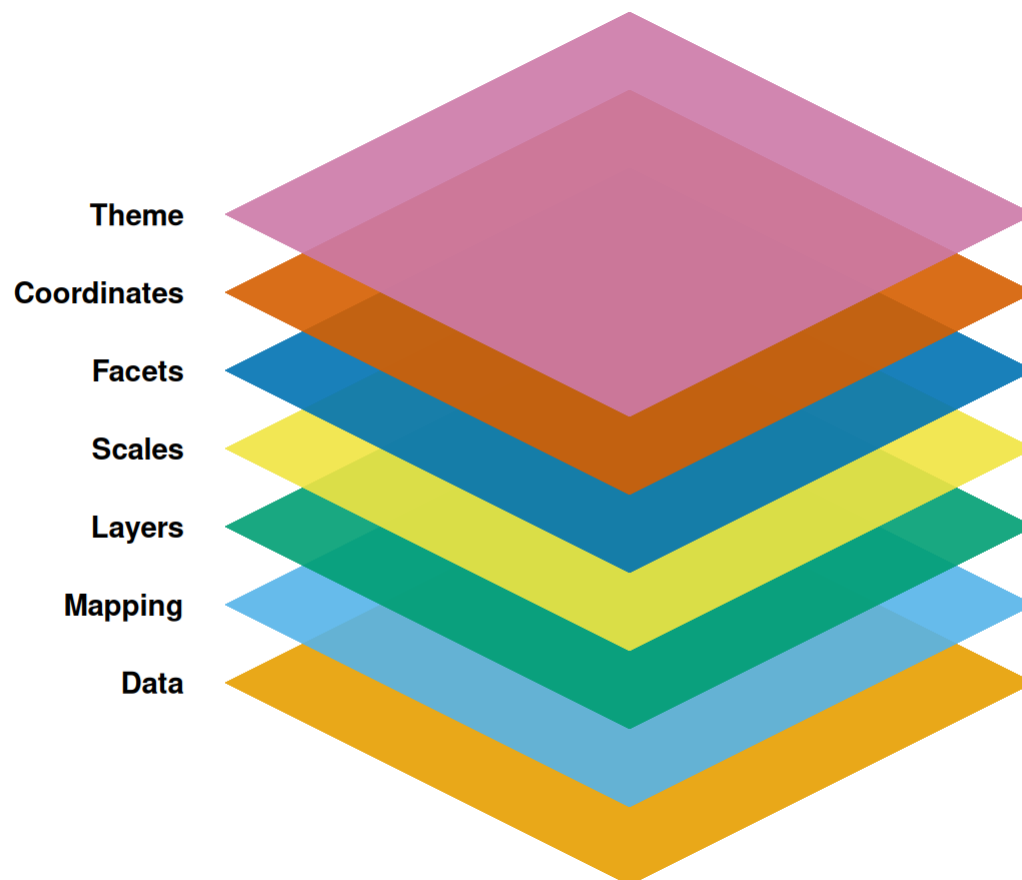
After completing the experiment, students will be able to –

- Create professional layered visualizations
- Apply data-driven aesthetics
- Interpret multi-layer pots
- Customize themes for presentation quality graphs
- Prepare publication ready visualizations

**Theory:**

ggplot2 is an R package for producing visualizations of data. Unlike many graphics packages, ggplot2 uses a conceptual framework based on the grammar of graphics. This allows you to ‘speak’ a graph from composable elements, instead of being limited to a predefined set of charts

For structure, we go over the 7 composable parts that come together as a set of instructions on how to draw a chart



Out of these components, ggplot2 needs at least the following three to produce a chart: data, a mapping, and a layer. The scales, facets, coordinates, and themes have sensible defaults that take away a lot of finicky work

**Data**

As the foundation of every graphic, ggplot2 uses data to construct a plot. The system works best if the data is provided in a tidy format, which briefly means a rectangular data frame structure where rows are observations and columns are variables.

As the first step in many plots, you would pass the data to the `ggplot()` function, which stores the data to be used later by other parts of the plotting system. For example, if we intend to make a graphic about the mpg dataset, we would start as follows:

```
ggplot(data = mpg)
```

## Mapping

The mapping of a plot is a set of instructions on how parts of the data are mapped onto aesthetic attributes of geometric objects. It is the ‘dictionary’ to translate tidy data to the graphics system.

A mapping can be made by using the `aes()` function to make pairs of graphical attributes and parts of the data. If we want the `cty` and `hwy` columns to map to the x- and y-coordinates in the plot, we can do that as follows:

```
ggplot(mpg, mapping = aes(x = cty, y = hwy))
```

## Layers

The heart of any graphic is the layers. They take the mapped data and display it in something humans can understand as a representation of the data. Every layer consists of three important parts:

The geometry that determines how data are displayed, such as points, lines, or rectangles.

The statistical transformation that may compute new variables from the data and affect what of the data is displayed.

The position adjustment that primarily determines where a piece of data is being displayed.

A layer can be constructed using the `geom_*`() and `stat_*`() functions. These functions often determine one of the three parts of a layer, while the other two can still be specified. Here is how we can use two layers to display the `cty` and `hwy` columns of the `mpg` dataset as points and stack a trend line on top.

```
ggplot(mpg, aes(cty, hwy)) +  
  # to create a scatterplot  
  geom_point() +  
  # to fit and overlay a loess trendline  
  geom_smooth(formula = y ~ x, method = "lm")
```

## Scales

Scales are important for translating what is shown on the graph back to an understanding of the data.

The scales typically form pairs with aesthetic attributes of the plots, and are represented in plots by guides, like axes or legends. Scales are responsible for updating the limits of a plot, setting the breaks, formatting the labels, and possibly applying a transformation.

To use scales, one can use one of the scale functions that are patterned as `scale_{aesthetic}_{type}()` functions, where {aesthetic} is one of the pairings made in the mapping part of a plot. To map the `class` column in the `mpg` dataset to the `viridis` colour palette, we can write the following:

```
ggplot(mpg, aes(cty, hwy, colour = class)) +  
  geom_point() +  
  scale_colour_viridis_d()
```

## Facets

Facets can be used to separate small multiples, or different subsets of the data. It is a powerful tool to quickly split up the data into smaller panels, based on one or more variables, to display patterns or

trends (or the lack thereof) within the subsets.

The facets have their own mapping that can be given as a formula. To plot subsets of the mpg dataset based on levels of the drv and year variables, we can use `facet_grid()` as follows:

```
ggplot(mpg, aes(cty, hwy)) +
  geom_point() +
  facet_grid(year ~ drv)
```

## Coordinates

You can view the coordinates part of the plot as an interpreter of position aesthetics. While typically Cartesian coordinates are used, the coordinate system powers the display of map projections and polar plots.

We can also use coordinates to display a plot with a fixed aspect ratio so that one unit has the same length in both the x and y directions. The `coord_fixed()` function sets this ratio automatically.

```
ggplot(mpg, aes(cty, hwy)) +
  geom_point() +
  coord_fixed()
```

## Theme

The theme system controls almost any visuals of the plot that are not controlled by the data and is therefore important for the look and feel of the plot. You can use the theme for customizations ranging from changing the location of the legends to setting the background color of the plot. Many elements in the theme are hierarchical in that setting the look of the general axis line affects those of the x and y axes simultaneously.

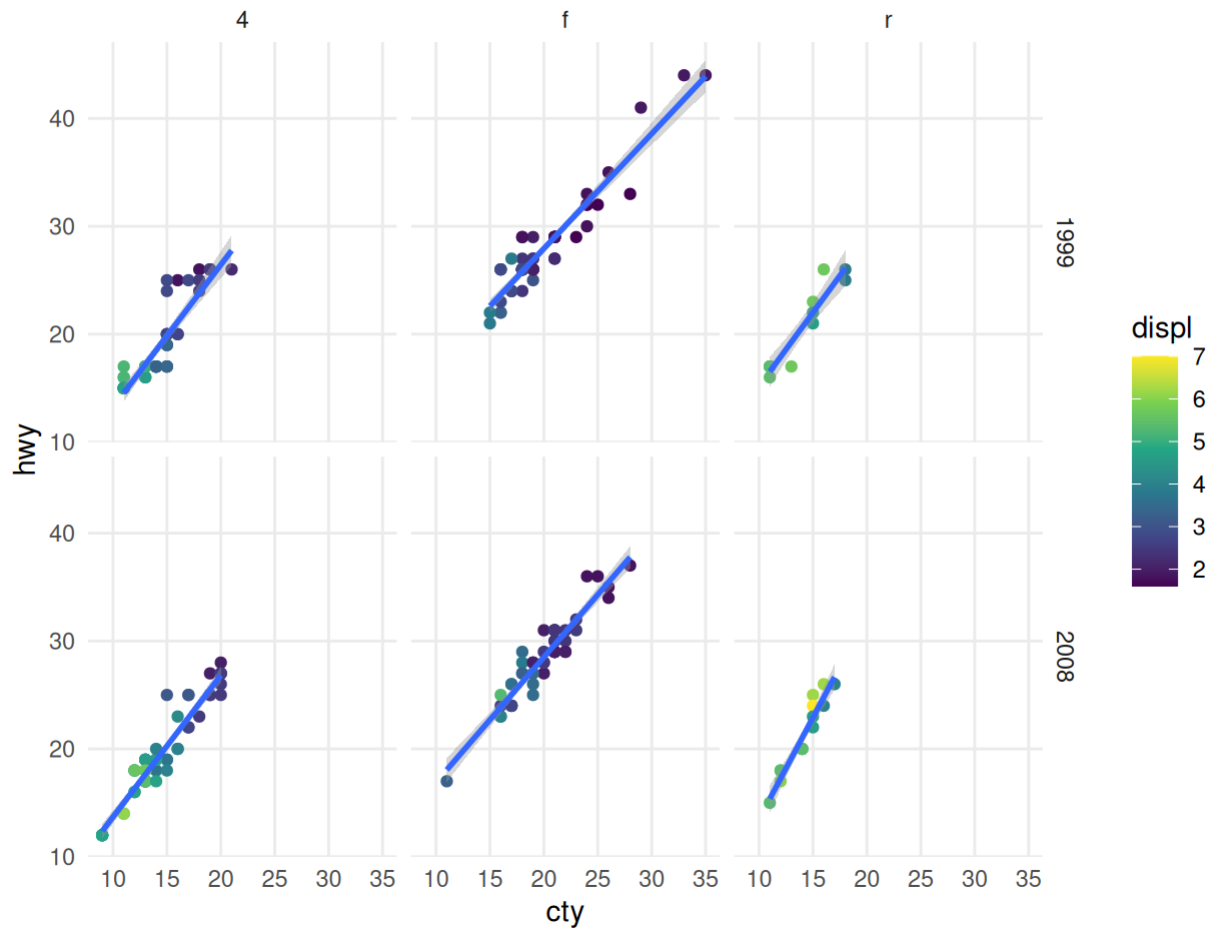
To tweak the look of the plot, one can use many of the built-in `theme_*`() functions and/or detail specific aspects with the `theme()` function. The `element_*`() functions control the graphical attributes of theme components.

```
ggplot(mpg, aes(cty, hwy, colour = class)) +
  geom_point() +
  theme_minimal() +
  theme(
    legend.position = "top",
    axis.line = element_line(linewidth = 0.75),
    axis.line.x.bottom = element_line(colour = "blue")
  )
```

As mentioned at the start, you can layer all of the pieces to build a customized plot of your data, like the one shown at the beginning of this vignette:

```
ggplot(mpg, aes(cty, hwy)) +
  geom_point(mapping = aes(colour = displ)) +
  geom_smooth(formula = y ~ x, method = "lm") +
```

```
scale_colour_viridis_c() +
facet_grid(year ~ drv) +
coord_fixed() +
theme_minimal() +
theme(panel.grid.minor = element_blank())
```



### Task to Perform

#### Sales performance analysis of a retail company using layered ggplot2 visualizations

A retail company wants to analyze the sales performance of its employees. The management is interested in:

- Comparing sales across different regions
- Identifying trends between experience and sales
- Understanding performance distribution
- Highlighting top-performing regions

You are hired as Data Analyst to create layered visualizations using ggplot2 to support business decisions

Dataset to use to perform this experiment –

```
sales_data = data.frame(
  Employee_ID = 1:20,
  Region =
c("North", "South", "East", "West", "North", "South", "East", "West",
"North", "South", "East", "West", "North", "South", "East", "West",
  "North", "South", "East", "West"),
  Experience = c(1, 2, 3, 4, 2, 5, 3, 6, 4, 3, 5, 2, 6, 4, 3, 5, 7, 2, 6, 4),
  Sales = c(200, 250, 300, 280, 220, 350, 330, 400, 310, 290,
360, 240, 420, 370, 340, 390, 450, 260, 410, 300)
)
```

### Task 1 –

#### Create a layered scatter plot:

- X-axis: Experience
- Y-axis: Sales
- Color by region
- Add a (regression or mean) line
- Apply a professional theme

### Task 2 –

#### Create a layered box plot –

- Compare sales by region
- Use fill aesthetics
- Apply minimal theme

### Task 3 –

#### Create a histogram with density layer for sales distribution

### Observations:

- Layering allows multiple visual insights in one plot
- Aesthetic mapping improves readability
- Regression line shows trend of performance
- Themes significantly improve visual appeal
- ggplot2 plots are more professional than base R plots

### Conclusion:

The experiment demonstrates the power of layered visualization using ggplot. By separating data, aesthetics and geometric layers, students can build flexible and customizable plots. The ability to apply themes enhances presentation quality, making ggplot2 suitable for research and industry analytics

**Expected Oral Questions:**

- 1) What is Grammar of Graphics?
- 2) What is aesthetic mapping?
- 3) Difference between `geom_point()` and `geom_line()`?
- 4) What is layering in `ggplot2`?
- 5) Difference between `theme_minimal()` and `theme_classic()`?
- 6) What is the use of `alpha` in aesthetics?
- 7) How to remove legend?
- 8) What is the difference between `color` and `fill`?
- 9) Why is `ggplot2` preferred in industry?

**FAQs in Interview:**

- 1) Why is mapping vs setting aesthetics?
- 2) Why use `ggplot2` instead of base R?
- 3) How do you create multi-layer visualization?
- 4) How to apply custom theme?
- 5) How do you add statistical transformation?